

Livrable2 - BDD

Batist AUREILLE - Info 4A FISA

Phase 2 : Migration MongoDB

1 Introduction

Dans cette deuxième partie, l'objectif était de réaliser une migration de notre base SQL vers une base MongoDB afin d'exporter notre projet vers un langage NoSQL. Il fallait ensuite comparer les performances de nos 9 requêtes entre ces 2 méthodes puis structurer nos données par film dans une collection, qui se veut être plus simple d'accès que les bases SQL classiques.

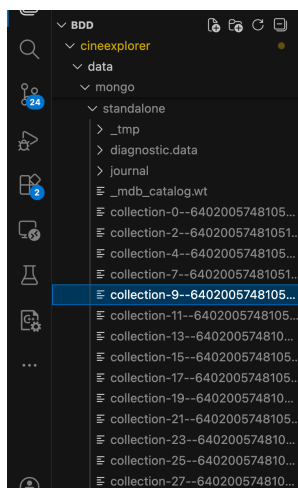
2 T2.1 Installation et Configuration

Après avoir suivi la méthode indiquée dans le sujet du projet, on est en mesure de lancer un serveur Mongo DB sur lequel on va migrer l'ensemble de nos données réparties dans nos différentes tables SQL. On peut voir la version de Mongo DB utilisée ici, peut-être pas la dernière version en raison de la qualité de mon PC qui est un peu ancien et réticent avec les nouvelles versions de nombreux outils.

```
PROBLÈMES 1 SORTIE PORTS CONSOLE DE DÉBOGAGE TERMINAL ... zsh + v [ ] [ ] ... | [ ] [ ] X
● (base) batistaureille@eduroam-2-211-150 BdD % python3 -c "from pymongo import MongoClient; print(f'Ve
rsion MongoDB : {MongoClient('mongodb://localhost:27017/').server_info()['version']}')"
Version MongoDB : 7.0.5
○ (base) batistaureille@eduroam-2-211-150 BdD %
```

3 T2.2 Migration des collections plates

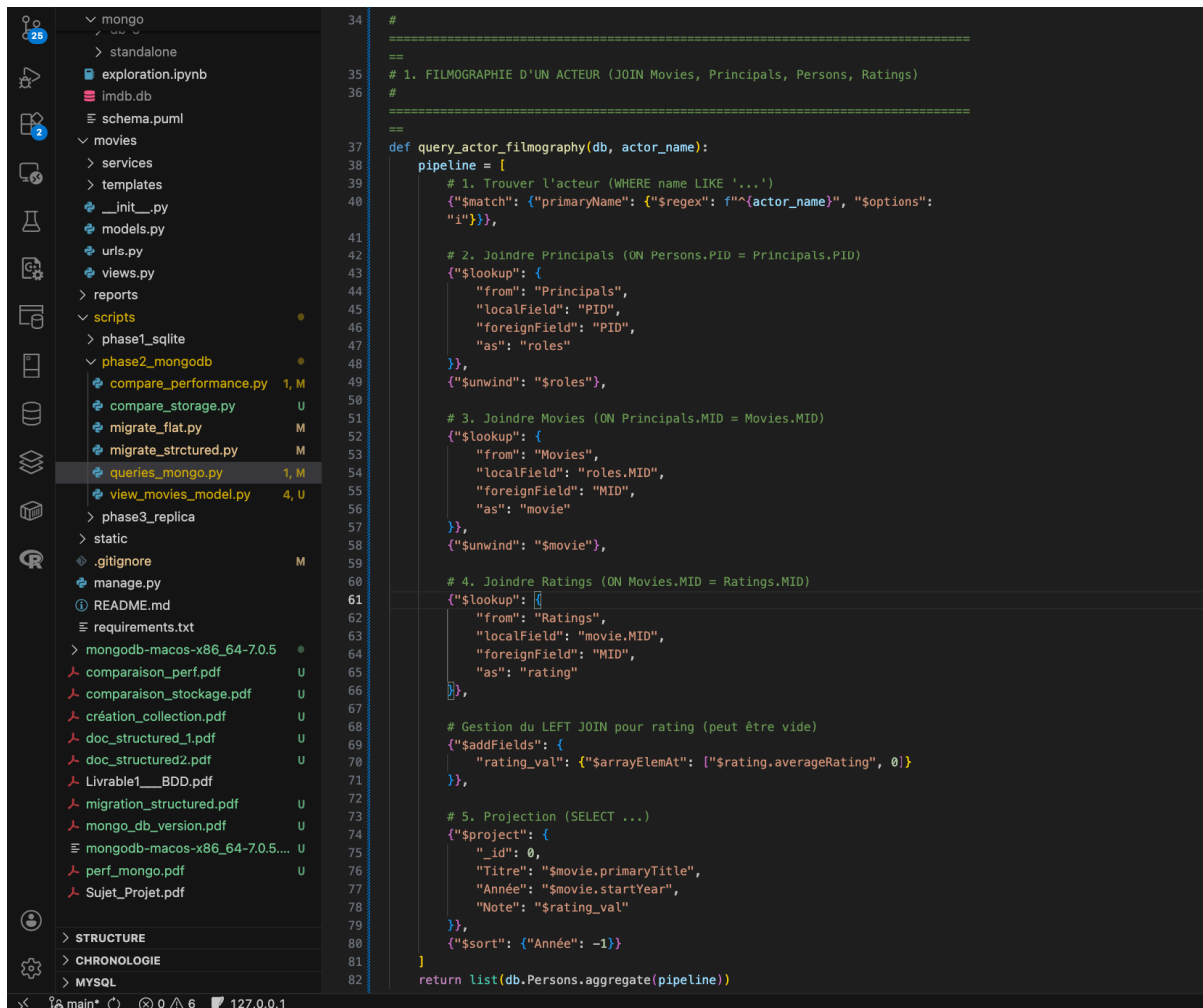
Le fichier migrate_flat.py permet de récupérer les données de nos tables SQL afin de créer des collections correspondantes. Il fallait être vigilant et bien respecter le fait d'extraire les données de la base SQL et pas des csv. Lorsque le script a fini de tourner, on en arrive à presque 12 millions de données importées, ce qui correspond à la quantité de données qu'on avait dans les différentes bases SQL dans la partie 1. Le script permet de créer automatiquement de multiples collections dans la rubrique standalone de notre dossier data.



4 T2.3 Requêtes MongoDB équivalentes

La syntaxe pour les requêtes en MongoDB est différente de celle utilisée pour le requêtage SQL, nous nous sommes aidés d'un LLM pour comprendre la structure et faciliter l'écriture des résultats dans le terminal pour suivre le bon fonctionnement de nos requêtes. Comme indiqué, nous avons utilisé les outils tels que \$lookup ou \$group pour parvenir à nos fins. A noter que nous avons au préalable créé des index, notamment pour utiliser \$lookup, sinon les requêtes seraient beaucoup trop lourdes pour s'effectuer efficacement dans un temps raisonnable.

Voici l'équivalent de la requête 1 suivi des temps chronométrés pour chacune d'entre elles (vous retrouverez l'ensemble des autres requêtes dans le script dédié) :



```
34 #
35 ==
36 # 1. FILMOGRAPHIE D'UN ACTEUR (JOIN Movies, Principals, Persons, Ratings)
37 ==
38 def query_actor_filmography(db, actor_name):
39     pipeline = [
40         # 1. Trouver l'acteur (WHERE name LIKE '...')
41         {"$match": {"primaryName": {"$regex": f"^{actor_name}", "$options":
42             "i"}}},
43         # 2. Joindre Principals (ON Persons.PID = Principals.PID)
44         {"$lookup": {
45             "from": "Principals",
46             "localField": "PID",
47             "foreignField": "PID",
48             "as": "roles"
49         }},
50         {"$unwind": "$roles"},
51         # 3. Joindre Movies (ON Principals.MID = Movies.MID)
52         {"$lookup": {
53             "from": "Movies",
54             "localField": "roles.MID",
55             "foreignField": "MID",
56             "as": "movie"
57         }},
58         {"$unwind": "$movie"},
59         # 4. Joindre Ratings (ON Movies.MID = Ratings.MID)
60         {"$lookup": {
61             "from": "Ratings",
62             "localField": "movie.MID",
63             "foreignField": "MID",
64             "as": "rating"
65         }},
66         # Gestion du LEFT JOIN pour rating (peut être vide)
67         {"$addFields": {
68             "rating_val": {"$arrayElemAt": ["$rating.averageRating", 0]}
69         }},
70         # 5. Projection (SELECT ...)
71         {"$project": {
72             "_id": 0,
73             "Titre": "$movie.primaryTitle",
74             "Année": "$movie.startYear",
75             "Note": "$rating_val"
76         }},
77         {"$sort": {"Année": -1}}
78     ]
79
80 return list(db.Persons.aggregate(pipeline))
81
82
```

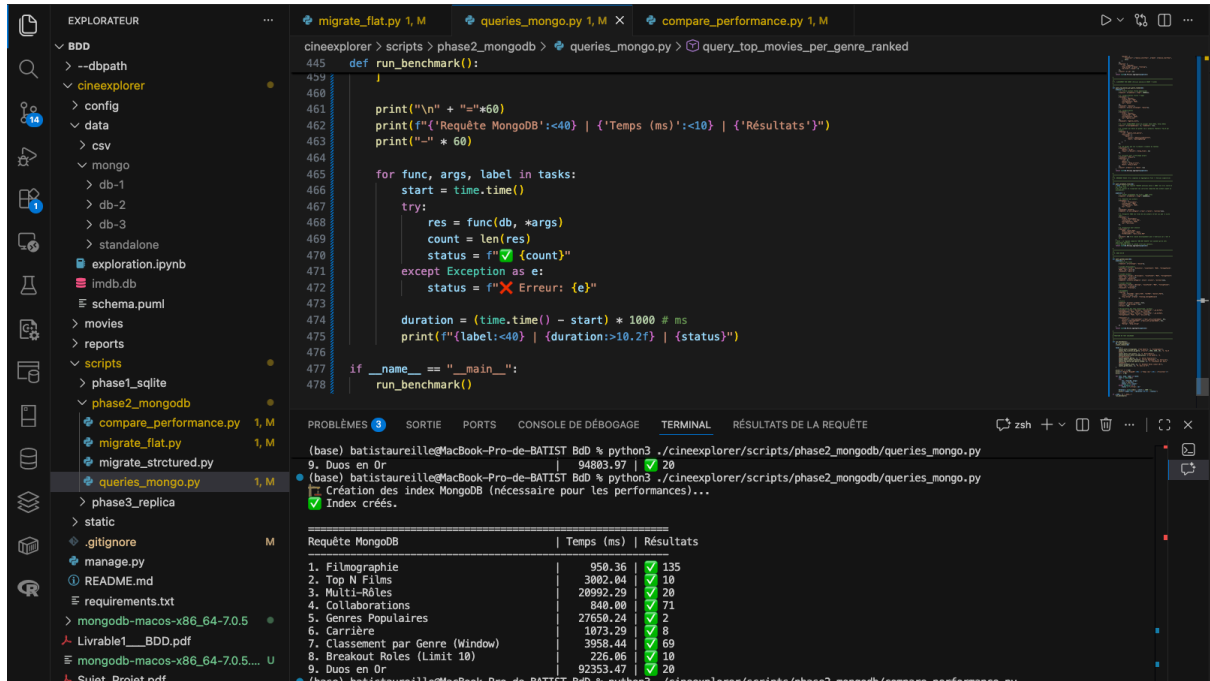


Table 1: Comparaison des temps d'exécution (MongoDB indexé vs. SQL indexé)

Requête	MongoDB (ms)	SQL (ms)	Gain de Performance (%)
query_actor_filmography	950.36	5250.58	+81.90%
query_director_actor_collaborations	840.00	78.28	-973.07%
query_top_n_movies_by_genre	3002.04	5419.72	+44.61%
query_multi_role_actors	20992.29	54.91	-38 130.36%
query_popular_genres	27650.24	315.81	-8 655.34%
query_actor_career_evolution	1073.29	5122.73	+79.05%
query_top_movies_per_genre_ranked	3958.44	178.34	-2 119.59%
query_breakout_roles	226.06	2868.47	+92.12%
query_golden_duos	92353.47	11034.97	-736.92%

Note : Un pourcentage négatif indique que MongoDB est plus lent que SQL (régression).

Explications des résultats :

Les victoires de MongoDB (en positif) concernent les requêtes de lecture directe ou de filtrage simple (actor_filmography, breakout_roles), confirmant son efficacité pour récupérer des documents complets sans jointures.

Les défaites massives (en négatif) (ex: -38 000% pour multi_role_actors) s'expliquent par la difficulté de MongoDB à effectuer des opérations analytiques complexes (jointures multiples, group by complexes) qui sont natives et optimisées en SQL.

5 T2.4 Documents structurés

Il s'agit de la partie la plus importante de cette phase 2. Le but est de créer des documents structurés permettant d'avoir accès à toutes les infos d'un film en 1 seule requête. Le script migrate.strctured.py (erreur dans le nom du fichier conservée...) permet de créer ces documents à partir de l'ensemble des collections importées, il crée alors une nouvelle collection appelée movies_complete_ALL. Comme suggérée, la méthode que l'on a utilisé est le pipeline d'agregation avec \$lookup multiples. Ce script a mis un temps conséquent à s'exécuter en intégralité, de l'ordre de 30min. Il a permis de créer environ 220k documents structurés correspondant chacun à 1 film en particulier.

dance logique des données (duplication des noms d'acteurs, genres, etc.), la taille physique sur le disque diminue de 44%. Cela s'explique par la suppression de la surcharge administrative (overhead) de millions de petits documents (notamment dans la collection de jointures 'Principals') et par l'efficacité de la compression native de MongoDB qui gère très bien les données redondantes. (Ecrit rédigé par un LLM après consultation de plusieurs sites explicatifs)" Cela semble donc être une optimisation automatique réalisée par MongoDB qui permet de ne pas augmenter très fortement le stockage dans un cas comme celui-ci.

Voici un tableau récapitulatif de la complexité selon les actions à réaliser :

Action	Complexité SQL (Relationnel)	Complexité MongoDB (Document)	Pourquoi ?
LIRE (GET)	● Haute	● Très Basse	MongoDB n'a pas besoin de "recoller" les données (JOINS).
ÉCRIRE (POST)	● Moyenne	● Basse	En SQL il faut insérer dans 4 tables + gérer les clés étrangères (Transaction). En Mongo, on insère 1 seul gros JSON.
MODIFIER (UPDATE)	● Très Basse	● Haute	En SQL, on modifie une seule ligne (Source unique de vérité). En Mongo, il faut modifier N documents (Redondance).
ÉVOLUER (ALTER)	● Haute	● Basse	Ajouter un champ en SQL demande un ALTER TABLE qui peut bloquer la base. En Mongo, on ajoute juste le champ dans le JSON.

5.1 Analyse documents plats vs structurés

Voici un tableau synthétique permettant de comparer efficacement ces 2 méthodes :

Critère	Documents Plats	Documents Structurés
Principe	Normalisation (éviter la redondance)	Dénormalisation (optimiser la lecture)
Requête type	SELECT ... JOIN ... JOIN ...	db.collection.find_one(...)
Nombre de requêtes	N requêtes (ou 1 très complexe)	1 requête simple
Vitesse d'affichage	Lente (CPU intensif)	Instantanée
Vitesse d'écriture	Rapide (petits ajouts)	Plus lente (gros documents)
Mise à jour globale	Simple (1 ligne à changer)	Complexe (N documents à changer)

6 Conclusion

Le passage d'un modèle plat à un modèle structuré représente un changement de paradigme : nous acceptons de complexifier les écritures et les mises à jour pour offrir une performance de lecture maximale et une simplification drastique du code applicatif. Les résultats empiriques montrent non seulement un gain de performance, mais aussi, contre toute attente, une réduction du volume de stockage grâce à l'élimination des métadonnées superflues.

Puisque IMDb est un site de consultation (on lit les fiches films 99% du temps, on modifie rarement le casting d'un vieux film), la complexité réduite du code de lecture en MongoDB est un immense avantage, malgré la difficulté des mises à jour.

NB : L'utilisation d'un LLM a été très utile ici afin de comprendre comment nous devons structurer nos différents codes de migration, que nous n'avions jamais réalisé jusqu'à lors. Il a permis de simplifier et dynamiser les affichages de résultats afin de mieux comprendre nos points d'étapes.